

Spike: Task 7**Title:** Performance Measurement**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is to demonstrate my ability to collect and analyse software performance data while applying my knowledge in comparing and evaluating functions, IDE and compiler settings.

Besides this report, what else was created?

- Code see \COS30031-2023-102608927\07 - Spike - Performance Measurement\Task7\Task7

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Lecture 3.1 Code Performance Notes.

Tasks undertaken:

- Download and install Visual Studio
- Download and install Visual Studio C++ Debugging Packages
- Create and Configure VS Project File with a .cpp file to build.
- Debug Run Tests such as
 - Single Tests
 - Ramp Up Tests
 - Repeatability
 - Function Comparisons
 - IDE Settings
 - Compiler Settings

What we found out:

Single Tests:

```
1 int count_char_using_find_first_of(string s, char delim)
2 {
3     int count = 0;
4     // note: string::size_type pos = s.find_first_of(delim);
5     auto pos = s.find_first_of(delim);
6     while ( (pos = s.find_first_of(delim, pos)) != string::npos)
7     {
8         count++;
9         pos++;
10    }
11    return count;
12 }
13
14 // - count char using fast std::count
15 int count_char_using_count(string s, char delim)
16 {
17     return count(s.begin(), s.end(), delim);
18 }
```

Using sample code 1 to measure the execution times of the individual functions such as “count_char_using_find_first_of” which repeats the “find_first_of” method to count occurrences of a character in a string and gives a result and “count_char_using_count” to count occurrences using std::count making both functions similar in execution time based on the following modifications I made to sample code 1 main function where I wanted to print more information regarding the execution time.

```
1 int main()
2 {
3     // Simple wrapper around a linear set of time tests
4     linear_rampup_test();
5
6     // Simple wrapper around an exponential set of time tests
7     exponential_rampup_test();
8
9     // Compare the two different methods of counting in a string
10    // - show result in nanoseconds?
11    string s1 = "This is a really simple string but it will do for testing.";
12
13    // Timing for count_char_using_find_first_of
14    auto start1 = steady_clock::now();
15    int result1 = count_char_using_find_first_of(s1, 's');
16    auto end1 = steady_clock::now();
17    duration<double> diff1 = end1 - start1;
18    cout << "count_char_using_find_first_of time: " << diff1.count() << " s" << endl;
19    cout << "result: " << result1 << endl;
20
21    // Timing for count_char_using_count
22    auto start2 = steady_clock::now();
23    int result2 = count_char_using_count(s1, 's');
24    auto end2 = steady_clock::now();
25    duration<double> diff2 = end2 - start2;
26    cout << "count_char_using_count time: " << diff2.count() << " s" << endl;
27    cout << "result: " << result2 << endl;
28
29 }
30 }
```

Function	Test 1 (in microseconds)	Test 2 (in microseconds)	Test 3 (in microseconds)
count_char_using_find_first_of	3.8 µs	3.9 µs	4.2 µs
count_char_using_count	30.3 µs	15.9 µs	16.1 µs

As you can see there is a difference between execution times for these two functions which could be affected by various factors such as algorithmic complexity and CPU processing.

Ramp Up Tests:

```

1 void exponential_rampup_test()
2 {
3     cout << " << Exponential Ramp-up Test >> " << endl;
4     int total;
5     // ull (suffix) == "unsigned long long" in c
6     for (auto size = 1ull; size < 1000000000ull; size *= 100)
7     {
8         // 1. get start time
9         auto start = steady_clock::now();
10        // 2. do some work (create, fill, find sum)
11        vector<int> v(size, 42);
12        total = accumulate(v.begin(), v.end(), 0u);
13        // 3. show duration time
14        auto end = steady_clock::now();
15        duration<double> diff = end - start;
16        cout << " - size: " << size << ", time: " << diff.count() << " s";
17        cout << ", time/int: " << diff.count() / size << "s/int" << endl;
18
19        // TIP: time in nanoseconds? Cast result of chrono::duration.count() ...
20        // auto _dur = duration_cast<nanoseconds>( end - start ).count();
21        // cout << _dur << endl;
22    }
23    cout << "done." << endl;
24 }
25
26 void linear_rampup_test()
27 {
28     cout << " << Linear Ramp-up Test >> " << endl;
29     int total;
30     for (auto size = 1; size <= 5; size += 1)
31     {
32         int vec_size = size * 10000;
33         // 1. get start time
34         auto start = steady_clock::now();
35         // 2. do some work (create, fill, find sum)
36         vector<int> v(vec_size, 42);
37         // std::accumulate (from <numeric>) collects from begin, to end
38         // - in this case (default) it is the sum total of all the values in v
39         total = accumulate(v.begin(), v.end(), 0u);
40         // 3. show duration time
41         auto end = steady_clock::now();
42         duration<double> diff = end - start;
43         cout << " - size: " << vec_size << ", time: " << diff.count() << " s";
44         cout << ", time/int: " << diff.count() / vec_size << "s/int" << endl;
45     }
46     cout << "done." << endl;
47 }

```

For ramp up tests we can use these two functions in sample 1 code to measure the execution time based on the different input sizes we have given it. Linear Ramp Up incrementally increases the size of the vector by 10,000 elements on each iteration while exponential ramp up function increases the size of the vector exponentially by 100 times each iteration.

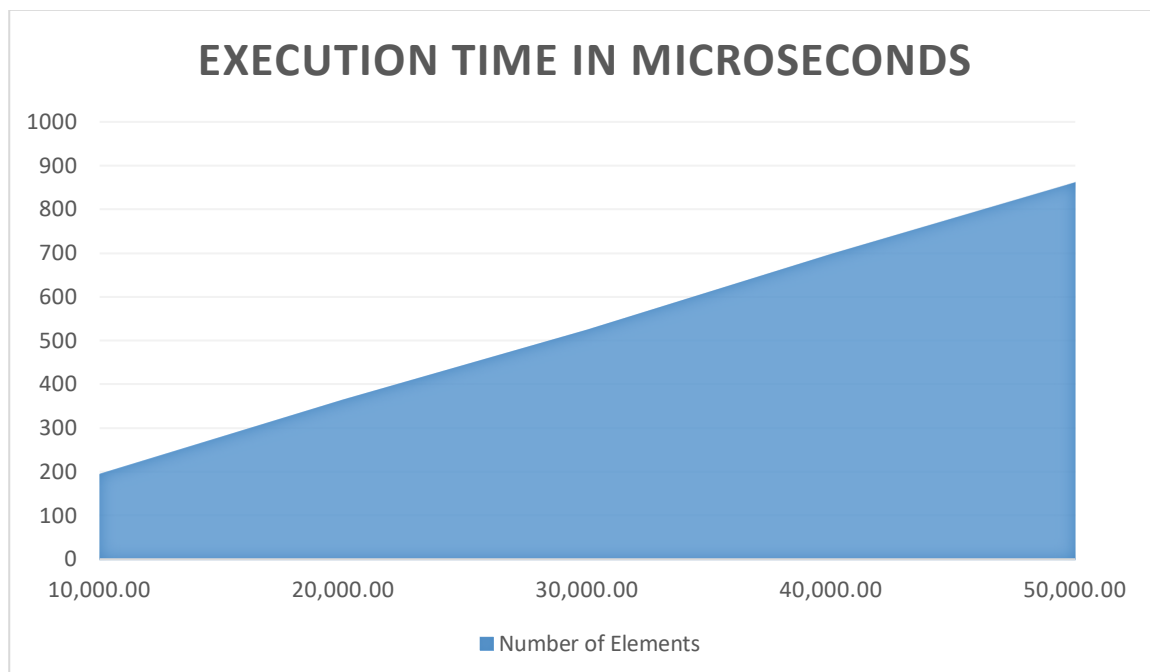
Linear Ramp-Up Testing :

Vector Size	Test 1 (in microseconds)	Test 2 (in microseconds)	Test 3 (in microseconds)
10,000	196.1	289.5	190.9
20,000	365.9	361.1	372.6
30,000	525.6	518.1	758.3
40,000	697.1	715.3	723.6
50,000	861.3	861.1	1,164.5

Exponential Ramp-Up Testing :

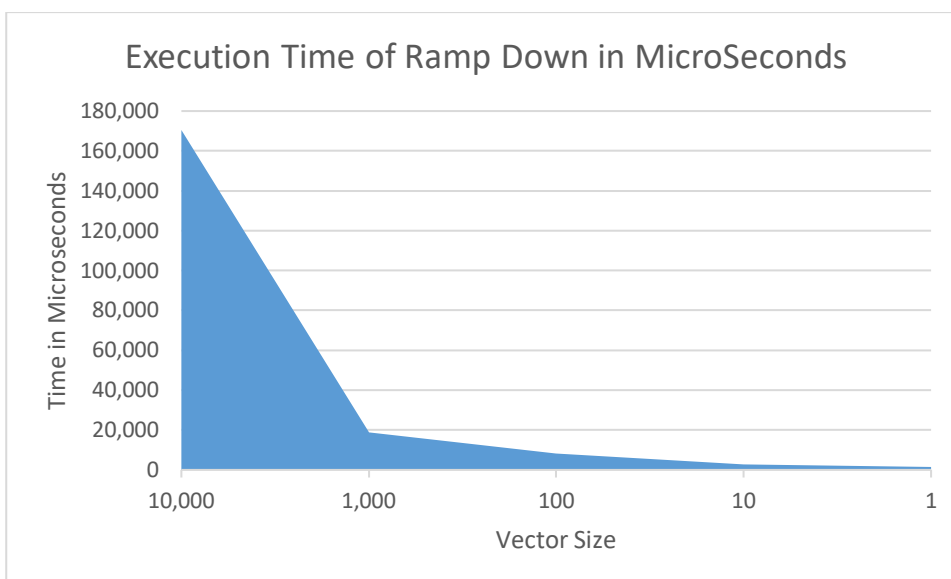
Vector Size	Test 1 (in microseconds)	Test 2 (in microseconds)	Test 3 (in microseconds)
1	3.8	3.7	4.6
100	4.8	4.6	7.1
10,000	191,200	189,300	190,700
1,000,000	17,653,800	17,662,800	17,677,000
100,000,000	2,039,293,900	2,050,560,200	2,074,923,900

Based on these test numbers lets focus only on linear ramp up timings as it shows a steady variation between tests using a chart and one of its test results.



```
1 void exp_rampdown_test(int steps, bool do_more)
2 {
3     cout << " << Particle Ramp-down Test >> " << endl;
4     // Find top exponent size value, so we can work down.
5     // No in-built int power() for C++/std, so ... roll our own.
6     uint size = 1;
7     for (int i = 1; i < steps; i++) size = size * 10;
8
9     // Do the test of vector creation, work on vector objects ...
10    for (auto step = steps; step > 0; step--)
11    {
12        // 1. get start time
13        auto start = steady_clock::now();
14        // 2. do some work
15        vector<ParticleClassA> v(size);
16        // if (do_more)
17        // {
18        //     for (auto &p: v)
19        //     {
20        //         p.do_stuff(step);
21        //     }
22        // }
23        // 3. show duration time
24        auto end = steady_clock::now();
25        auto _dur = duration_cast<nanoseconds>( end - start ).count();
26        cout << " - size: " << size << ", time: " << _dur << " ns";
27        cout << " (ns/count): " << _dur/size << endl;
28        // work down (by factors of 10 ...)
29        size = size / 10;
30    }
31    cout << "done." << endl;
32 }
```

Vector Size	10,000	1,000	100	10
Run 1 (µs)	170,500	18,900	8,100	2,700
Run 2 (µs)	173,400	18,400	6,600	2,100
Run 3 (µs)	170,800	18,600	3,700	2,200



In sample 2 code function “exp_rampdown_test” function starts execution from the largest vector to smallest vector and based on the results we can see that it is faster due to various factors that could have an impact on execution time such as cache hits and memory allocation or deallocation.

Repeatability:

When talking about repeatability in context to sample code 1 and sample code 2 functions and its results for time taken to execute, we see that sometimes consistency of the outcome varies when repeatedly running the code multiple times this could be due to many variables such as the machine it's running could be using too many resources in the background for example the results from “count_char_using_find_first_of” and “count_char_using_count” where there was variation in execution times across the 3 tests. Therefore, it is important to run these tests in a stable environment to have consistent and smaller variations when testing performance.

Function Comparisons:

Functions such as sample 1 code “count_char_using_find_first_of” where it uses repeated string search to find occurrences in string characters by looping which leads to longer time complexity and while “count_char_using_count” uses `std::count` to tally the character occurrences which is more optimized for performance leads to better execution times as shown earlier in the results where “count_char_using_find_first_of” took 3.8 microseconds and “count_char_using_count” took 3.03 microseconds.

IDE Settings:

Using debug mode to execute the sample code 1 and 2 compared to release mode shows us the difference between execution times which is much slower due to no C++ optimization is enabled like in release mode also sometimes debug mode uses more resources for error checking and memory initialization compared to release mode.

Compiler Settings:

Using Visual Studio's compiler optimization settings is set to default for normal release mode while it's disabled for debug mode which does not allow us to compare execution times.